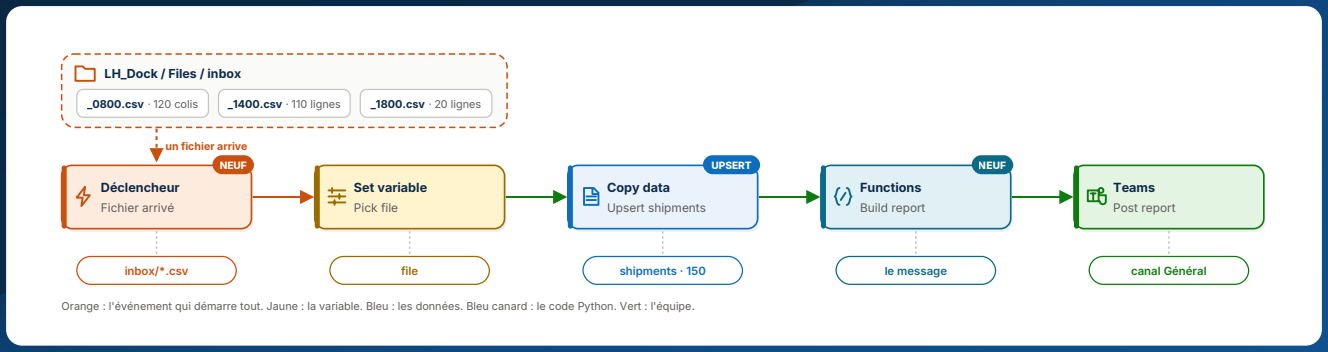
MICROSOFT FABRIC · DATA FACTORY · PIPELINE ÉVÉNEMENTIEL · GUIDE DE RÉFÉRENCE

Pipeline Microsoft Fabric : déclencher, fusionner, calculer, prévenir

DÉCLENCHER CHOISIR FUSIONNER CALCULER PRÉVENIR

Personne n'appuie sur Exécuter : un fichier arrive dans le lakehouse et le pipeline démarre seul. Il fusionne les colis par Upsert, sans jamais doubler une ligne, fait écrire le compte rendu par une fonction Python hébergée dans Fabric, et le poste dans Teams.



4	3	3	150	0
activités	nouveautés	vagues de colis	colis suivis	clic pour lancer

AUTEUR
Abdoul Hamid Diallo, Microsoft Data & AI Engineer

PRÉREQUIS
Un espace de travail Fabric sur une capacité, Microsoft Teams avec une équipe

FICHIERS DU PROJET
Trois fichiers de colis et expressions.txt, qui contient le code Python et toutes les expressions

DURÉE
Environ 40 minutes, en quatre étapes vérifiées une à une

VUE D'ENSEMBLE

Ce que vous construisez

Le quai de Metz reçoit trois fois par jour un fichier de suivi de colis : à 8 h les colis préparés, à 14 h les premiers départs et les nouveaux colis, à 18 h les livraisons. Chaque fichier mélange des colis nouveaux et des colis déjà connus dont le statut a changé. Le pipeline démarre dès qu'un fichier arrive, met à jour la table `shipments` ligne par ligne, et annonce la vague dans Teams. Trois nouveautés : **le déclencheur sur événement**, **la copie en Upsert**, **l'activité Functions**.



Orange : l'événement qui démarre tout. Jaune : la variable. Bleu : les données. Bleu canard : le code Python. Vert : l'équipe.

Les quatre activités et le déclencheur

ÉLÉMENT	TYPE	RÔLE
`TR_Dock_Inbox`	Déclencheur NEUF	Écoute `Files/inbox` : chaque fichier créé lance le pipeline et lui donne son nom
`Pick file`	Set variable	Prend le nom du fichier venu de l'événement, ou le paramètre si on lance à la main
`Upsert shipments`	Copy data UPSERT	Insère les nouveaux colis, met à jour les colis connus, d'après `shipment_id`
`Build report`	Functions NEUF	Appelle la fonction Python `build_report` : elle lit le nom de la vague et rédige le compte rendu
`Post report`	Microsoft Teams	Poste le compte rendu dans le canal de l'équipe

Les trois vagues

FICHIER	CONTENU	LIGNES
`…_0800.csv`	120 colis préparés	120
`…_1400.csv`	80 colis qui changent de statut, 30 nouveaux	110
`…_1800.csv`	20 colis expédiés qui passent à livré	20

Six colonnes : `shipment_id`, `client`, `destination`, `weight_kg`, `status`, `updated_at`. Statuts : `PREPARE`, `EXPEDIE`, `LIVRE`, `RETARD`.

Ce qui est révolutionnaire

PLUS D'HORAIRE Un pipeline planifié tourne même quand rien n'est arrivé, et rate ce qui arrive juste après. Ici, c'est l'arrivée du fichier qui lance le pipeline, à la seconde.

FUSIONNER, PAS EMPILER Upsert compare chaque ligne à la table par sa clé : nouvelle, elle s'ajoute ; connue, elle remplace l'ancienne. 260 lignes reçues, 150 colis en table.

DU PYTHON DANS LE PIPELINE La règle de rédaction vit dans une fonction hébergée par Fabric, testable seule et réutilisable par n'importe quel pipeline.

Le pipeline en une image : un quai de messagerie

La sonnette
Le déclencheur sonne dès qu'un camion dépose un bordereau dans la boîte.

Le chef de quai
Pick file prend le bordereau qui vient d'arriver, par son nom.

Le registre
Upsert met à jour la fiche de chaque colis, ou en ouvre une s'il est nouveau.

Le secrétaire
La fonction Python rédige le compte rendu de la vague.

Le haut-parleur
Teams annonce la vague à toute l'équipe.

ÉTAPE 1

Le lakehouse et la fonction Python

1 · OBJETS | 2 · CHOISIR ET FUSIONNER | 3 · CALCULER ET PRÉVENIR | 4 · DÉCLENCHER

1 LH_Dock

LAKEHOUSE

1. Dans votre espace de travail, **Nouvel élément**, **Lakehouse**, nom `LH_Dock`.
2. Dans **Fichiers**, **Nouveau sous-dossier** `inbox`.
3. Chargez dans `inbox` **seulement** `shipments_2026-10-14_0800.csv` et `shipments_2026-10-14_1400.csv`. Gardez le fichier de 18 h pour l'étape 4 : c'est lui qui déclenchera le pipeline.

NE CHARGEZ PAS 18 H MAINTENANT À l'étape 4, son arrivée dans `inbox` doit être l'événement qui lance tout. S'il est déjà là, il n'y a plus d'arrivée à attendre.

2 FN_Dock

FONCTIONS DE DONNÉES UTILISATEUR · NEUF

1. **Nouvel élément**, **Fonctions de données utilisateur** (User data functions), nom `FN_Dock`.
2. **Nouvelle fonction**. Remplacez tout le code de l'éditeur par le bloc ci-dessous, copié depuis `expressions.txt`.
3. **Publier**. Attendez la fin : une à deux minutes.
4. Testez-la seule : à droite de `build_report`, **Exécuter**, `fileName` = `shipments_2026-10-14_0800.csv`, `rowsRead` = `120`.

```
import fabric.functions as fn

udf = fn.UserDataFunctions()

@udf.function()
def build_report(fileName: str, rowsRead: int) -> str:
    stamp = fileName.split("_")[-1].replace(".csv", "")
    if len(stamp) == 4 and stamp.isdigit():
        wave = stamp[0:2] + " h " + stamp[2:4]
    else:
        wave = stamp
    if rowsRead >= 115:
        load = "vague lourde"
    else:
        load = "vague normale"
    return "Quai de Metz, vague de " + wave + " : " + str(rowsRead) + " colis fusionnés dans shipments (" + load + ")."
```

CE QUE FAIT LA FONCTION Elle prend le nom du fichier et le nombre de lignes lues, en paramètres sans tiret bas : `fileName`, `rowsRead`. Elle extrait l'heure de la vague (`0800` devient `08 h 00`), qualifie la vague (lourde dès 115 lignes) et rend une phrase prête à poster.

RÉSULTAT DU TEST

```
Quai de Metz, vague de 08 h 00 : 120 colis fusionnés dans shipments (vague lourde).
```

AVANT LE PIPELINE ☐ deux fichiers dans `inbox` ☐ `FN_Dock` publiée ☐ le test renvoie la phrase ☐ le fichier 18 h attend

ÉTAPE 2

Choisir le fichier, puis le fusionner par Upsert

1 · OBJETS | 2 · CHOISIR ET FUSIONNER | 3 · CALCULER ET PRÉVENIR | 4 · DÉCLENCHER



3 PL_Dock_Express, paramètre et variable

1. **Nouvel élément**, **Pipeline**, nom `PL_Dock_Express`. Fond du canevas, onglet **Paramètres**, **Nouveau** :

Nom	`file_name`
Type	`String`
Valeur par défaut	`shipments_2026-10-14_0800.csv`

2. Onglet **Variables**, **Nouveau** : `file`, type `String`.

4 Pick file — SET VARIABLE

1. **Définir une variable**, nom `Pick file`. Variable `file`, valeur :

```
@coalesce(pipeline()?.TriggerEvent?.FileName,
pipeline().parameters.file_name)
```

DEUX FAÇONS DE DÉMARRER, UN SEUL PIPELINE Lancé par le déclencheur, `TriggerEvent.FileName` porte le nom du fichier arrivé. Lancé à la main, il est vide : `coalesce` prend alors le paramètre. Le `?.` évite l'erreur quand l'événement n'existe pas.

5 Upsert shipments — COPY DATA · UPSERT

1. **Copier les données**, **Ajouter au canevas**, nom `Upsert shipments`, relié derrière `Pick file`.
2. **Source** : `LH_Dock`, racine **Fichiers**, dossier `inbox`, nom de fichier en contenu dynamique :

```
@{variables('file')}
```

Format **DelimitedText**, **Première ligne comme en-tête** coché.

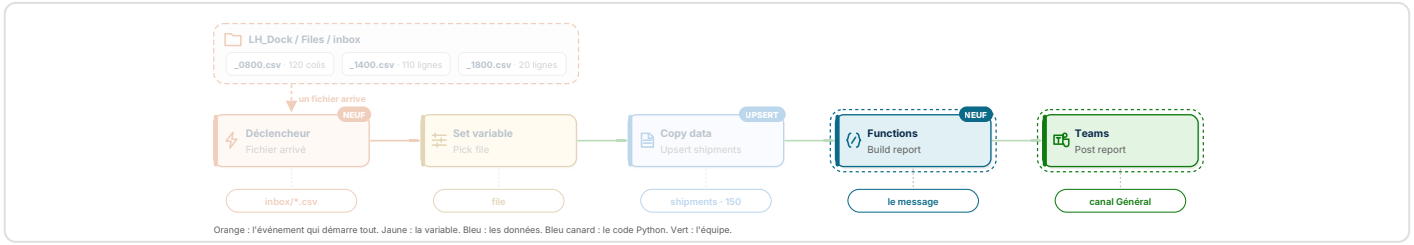
3. **Destination** : `LH_Dock`, racine **Tables**, nouvelle table `shipments`. **Action de table** : **Upsert**. **Colonnes clés** : `shipment_id`.
4. **Enregistrer**, **Exécuter** avec la valeur par défaut. Sortie : `rowsRead` = **120**.

CE QUE FAIT UPSERT Pour chaque ligne reçue, la table est interrogée par `shipment_id`. Absente, la ligne est insérée. Présente, elle est remplacée par la version reçue. C'est un `MERGE`, sans une ligne de SQL.

ÉTAPE 3

Faire rédiger le compte rendu par Python, le poster, rejouer

1 · OBJETS | 2 · CHOISIR ET FUSIONNER | 3 · CALCULER ET PRÉVENIR | 4 · DÉCLENCHER



6 Build report

FUNCTIONS · NEUF

1. **Activités**, **Functions** (Fonctions), nom `Build report`, relié derrière `Upsert shipments`.
2. **Paramètres** : type **Fonctions de données utilisateur Fabric**, votre espace de travail, élément `FN_Dock`, fonction `build_report`. Si une connexion est demandée : **Compte d'organisation**.
3. Les deux paramètres de la fonction apparaissent. Valeurs :

fileName · String	`@variables('file')`
rowsRead · Int	`@activity('Upsert shipments').output.rowsRead`

7 Post report

MICROSOFT TEAMS

1. **Microsoft Teams**, nom `Post report`, derrière `Build report`. Votre équipe, canal **Général**, **Titre** : `Quai de Metz`.
2. **Message** :

```
@{activity('Build report').output.output}
```

3. **Enregistrer**.

LA SORTIE D'UNE FONCTION Ce que `build_report` renvoie se lit dans `output.output` de l'activité. Vérifiez-le une fois dans **Sortie** de `Build report`.

Exécution 1 file_name = shipments_2026-10-14_0800.csv, à la main

Quatre activités vertes, **120** colis en table, tous `PREPARE`. Dans Teams : « Quai de Metz, vague de 08 h 00 : 120 colis fusionnés dans shipments (vague lourde). »

Exécution 2 file_name = shipments_2026-10-14_1400.csv, à la main

110 lignes lues, mais **150** colis en table : 80 mis à jour, 30 ajoutés. Dans Teams : « … vague de 14 h 00 : 110 colis fusionnés dans shipments (vague normale). »

Exécution 3 le même fichier de 14 h, une seconde fois

Toujours **150** colis. Upsert reconnaît chaque `shipment_id` et remplace la ligne par elle-même : rejouer un fichier ne double jamais rien.

ÉTAPE 4

Brancher le déclencheur, déposer un fichier, regarder

1 · OBJETS | 2 · CHOISIR ET FUSIONNER | 3 · CALCULER ET PRÉVENIR | 4 · DÉCLENCHER

8 TR_Dock_Inbox

DÉCLENCHEUR · NEUF

1. Dans `PL_Dock_Express`, ruban, **Déclencheur**, **Ajouter un déclencheur**. Source : **Événements OneLake**.
2. Choisissez `LH_Dock`, puis le dossier `inbox` : le chemin `Files/inbox` est filtré d'office. Type d'événement : seul `Microsoft.Fabric.OneLake.FileCreated` coché.
3. **Définir les filtres** :

Champ	`subject`
Opérateur	`La chaîne se termine par`
Valeur	`.csv`

4. **Suivant**, **Connecter**. Action : **Exécuter** `PL_Dock_Express`, règle nommée `TR_Dock_Inbox`, enregistrée dans un élément Activator. **Créer**.

LA RÈGLE EST-ELLE ALLUMÉE ? Ouvrez l'Activator, cliquez sur `TR_Dock_Inbox` : badge vert **Exécution**, bouton **Démarrer** grisé. Sinon, cliquez sur **Démarrer**.

CE QUI SE PASSE SOUS LE CAPOT OneLake publie un événement à chaque fichier créé. Activator le reçoit, applique le filtre, et lance le pipeline en lui transmettant l'événement : c'est lui que lit `pipeline()?.TriggerEvent`.

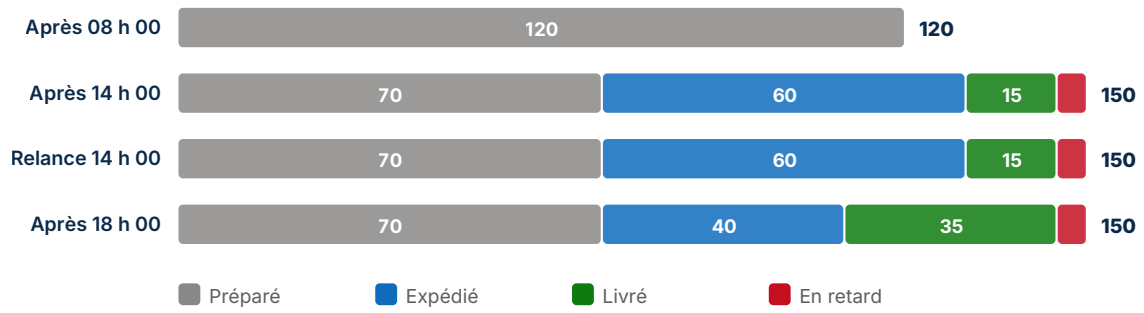
9 Exécution 4, sans cliquer sur Exécuter

1. Ouvrez `LH_Dock`, dossier `inbox`, **Charger des fichiers** : `shipments_2026-10-14_1800.csv`.
2. Dans l'Activator, onglet **Historique** de `TR_Dock_Inbox` : **1** activation aujourd'hui.
3. Barre de gauche, **...**, **Moniteur** : une exécution de `PL_Dock_Express` a démarré seule, état **Réussi**. Ne cliquez jamais sur **Exécuter** pour ce test.
4. Dans Teams, le compte rendu arrive, écrit par la fonction :

> AD Abdoul Hamid DIALLO 02:16
>
> Quai de Metz
>
> Quai de Metz, vague de 18 h 00 : 20 colis fusionnés dans shipments (vague normale).

LA PREUVE Vous n'avez rien lancé. Le nom de fichier n'a été tapé nulle part : il est venu de l'événement. Et la table compte toujours **150** colis, dont **35** livrés.

LA TABLE shipments APRÈS CHAQUE EXÉCUTION, PAR STATUT



Trois réflexes d'expert sur ce pipeline

Filtrer au plus près
Le dossier `inbox` et le filtre `.csv` évitent qu'un fichier écrit ailleurs, ou un fichier temporaire, ne lance le pipeline pour rien.

Une clé, et une seule
Upsert n'a de sens que si la clé est unique par ligne. Deux lignes du même colis dans un fichier : la dernière gagne, sans erreur.

Une fonction, mille pipelines
`build_report` se teste seule et se publie une fois. Tout pipeline de l'espace peut l'appeler, avec la même règle.

RÉFÉRENCE

Vérifier, retrouver une expression, débloquer

Vérifier dans le lakehouse

Ouvrez `LH_Dock`, **Point de terminaison SQL**, **Nouvelle requête SQL**, après l'exécution 4 :

```
SELECT status, COUNT(*) AS colis
FROM shipments
GROUP BY status ORDER BY status;

SELECT COUNT(*) AS colis,
  COUNT(DISTINCT shipment_id) AS ids
FROM shipments;
```

STATUS	COLIS
EXPEDIE	40
LIVRE	**35**
PREPARE	70
RETARD	5
Total, et ids distincts	**150 · 150**

260
lignes reçues sur les trois vagues

150
colis en table, chacun une seule fois

35
colis livrés après la vague de 18 h

0
clic pour lancer la dernière vague

Les expressions

OÙ	EXPRESSION
Pick file, variable `file`	`@coalesce(pipeline()?.TriggerEvent?.FileName, pipeline().parameters.file_name)`
Upsert shipments, fichier source	`@{variables('file')}`
Build report, `fileName`	`@variables('file')`
Build report, `rowsRead`	`@activity('Upsert shipments').output.rowsRead`
Post report, message	`@{activity('Build report').output.output}`

Si quelque chose bloque

SYMPTÔME	CAUSE ET CORRECTION
Publication de la fonction : « Erreur de syntaxe »	Un nom de paramètre contient un tiret bas. Les fonctions Fabric veulent `fileName` et `rowsRead`, jamais `file_name`.
« Fonctions de données utilisateur » absent	La fonctionnalité n'est pas activée dans le locataire. Remplacez `Build report` par une activité **Définir une variable** qui compose le message avec `concat()`.
L'action **Upsert** refuse une table qui n'existe pas	Premier passage en **Ajouter** avec le fichier de 8 h, puis repassez l'action en **Upsert** pour la suite.
La table double après la relance	L'action est restée sur **Ajouter**, ou la colonne clé n'est pas `shipment_id`.
Teams poste un message vide	La sortie de la fonction porte un autre nom. Ouvrez **Sortie** de `Build report` et reprenez le chemin exact dans le message.
Le dépôt du fichier ne lance rien	La règle est arrêtée, ou le filtre vise un autre dossier. Badge **Exécution** sur `TR_Dock_Inbox`, dossier `inbox`, événement FileCreated, puis redéposez le fichier.
Le pipeline démarre mais charge la vague de 8 h	L'événement n'a pas transmis le nom : dans l'action Activator, donnez au paramètre `file_name` la valeur du nom de fichier de l'événement.
Échec « AADSTS700082 », jeton d'actualisation expiré	Une connexion inutilisée 90 jours expire. **Paramètres**, **Gérer les connexions et les passerelles**, la connexion, **Modifier les informations d'identification**, **Compte d'organisation**, **Se connecter**. Ou créez une nouvelle connexion dans l'activité.
Suivant grisé dans l'assistant du déclencheur	Une ligne de filtre est vide. Remplissez-la (`subject`, se termine par, `.csv`) ou supprimez-la avec la corbeille.